

1. 컴퓨터 시스템 개요

컴퓨터의 기본 구조

- 주요 구성요소: 중앙처리장치, 기억장치, 입출력장치
- 입출력장치: 주변장치라고도 부름(Peripheral Device)
CPU는 Device Controller를 통해 동작 제어

어셈블리

- LOAD A, X ; 주소 X의 내용을 레지스터 A에 로드
- ADD A, Y ; 주소 Y의 내용을 레지스터 A의 값과 더하고 A에 적재
- STOR Z, A ; 주소 Z에 레지스터 A의 값 저장

기계어

연산코드	오퍼랜드
0 0 1	0 0 1 0 1

- 연산코드: 수행할 연산 지정
- 오퍼랜드: 연산에 사용될 데이터 / 주소
- 비트 수: 표현 가능한 최대 가지수에 연관

기억장치 저장

- Word 단위로 저장: CPU에 의해 한 번에 처리될 수 있는 비트 수

주소	값
0	00010010
1	10000011
2	11101010

8비트 시스템에서 데이터 저장:

- 주소 지정: Word / Byte 단위

시스템 버스와 장치 간 접속

- 주소 버스: 단방향
CPU의 데이터 조작 시에만 사용하므로
- 데이터, 제어 버스: 양방향
CPU와 다른 장치 모두 주고받을 데이터가 있으므로

- 기억장치 쓰기 동작

CPU가 발생:	주소 버스	쓰기 대상 주소
	데이터 버스	쓸 데이터
	제어 버스	쓰기 신호

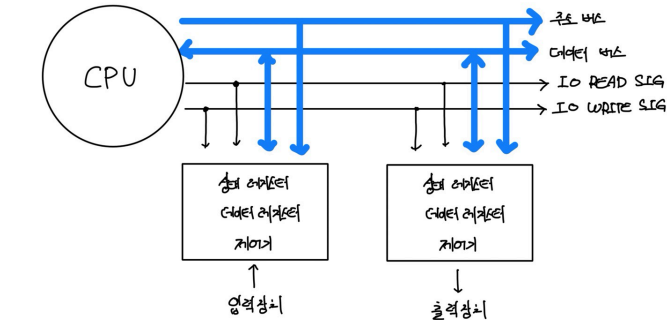
- 쓰기 시간 $t = \text{time_at}(\text{저장 완료}) - \text{time_at}(\text{CPU의 주소/데이터 발생})$

- 기억장치 읽기 동작

CPU가 발생:	주소 버스	읽기 대상 주소
	제어 버스	읽기 신호

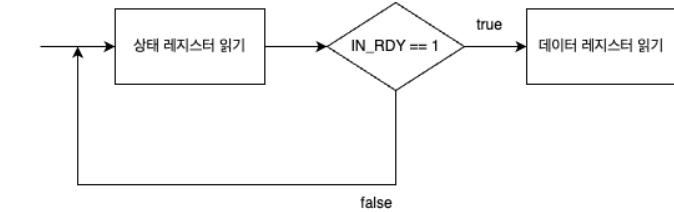
기억 장치가 발생: 데이터 버스 읽어낸 데이터

- 읽기 시간 $t = \text{time_at}(\text{데이터가 CPU에 도착}) - \text{time_at}(\text{CPU의 주소 발생})$



IO 장치 제어기

- CPU로부터 IO명령 받아서, 장치 제어, 데이터 이동
- 상태 레지스터: 장치의 현재 상태 나타내는 비트 저장
- 데이터 레지스터: CPU와 IO 장치 간 이동 데이터 임시 저장
- 키보드의 데이터 입력 과정
 - 키보드: 키 입력 시 대응 코드를 데이터 레지스터에 적재 $\Rightarrow \text{IN_RDY} = 1$
 - CPU:



- 프린터의 데이터 출력 과정
 - CPU:
 - 1. while(!OUT_RDY);
 - 2. 비트 검사 통과 시 프린트 데이터를 프린터 컨트롤러 내 데이터 레지스터에 저장
 - 프린터 컨트롤러:
 - 1. 데이터 레지스터 내용, 프린터로 전송
 - 2. 하드웨어 제어, 인쇄 수행

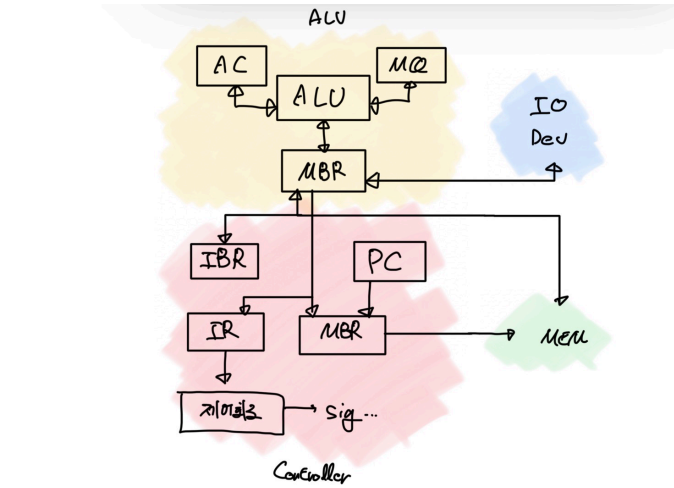
컴퓨터의 기본 기능

프로그램 실행 데이터 저장 데이터 이동 데이터 입출력 제어

컴퓨터 구조의 발전 과정

- 주요 부품들의 발전 과정
- 릴레이 $\rightarrow$ 진공관 $\rightarrow$ 트랜지스터 $\rightarrow$ IC
- 개선된 특징들: 처리속도 향상 / 저장용량 증가 / 크기 감소 / 가격 하락 / 신뢰도 향상

- 최초의 컴퓨터: 파스칼의 원형판
- Leibniz의 기계: 파스칼 계산기에 원형판 추가: 곱셈 / 나눗셈 추가
- Difference Engine
- Analytical Engine: 프로그래밍 가능, 프로그램 실행 순서 변경 가능(=조건 분기)
- ENIAC: 최초의 전자식 컴퓨터
 - 폰 노이만 구조 기반
 - 프로그램 + 데이터, 2진수 체계
- IAS 컴퓨터: 프로그램 저장 변경 가능
 - 프로그램 제어 유닛: 명령 Fetch/Decode
 - 산술 논리 연산 장치: ALU
 - 주기억장치: 명령어 + 데이터 저장
 - 입출력장치



주요 부품들의 발전

진공관 $\Rightarrow$ 트랜지스터 $\Rightarrow$ 집적회로(반도체)

- IC 사용
 - 통로 길이 감소 $\Rightarrow$ 동작 속도 상승
 - 컴퓨터 크기 감소
 - 회로 간 상호연결 $\Rightarrow$ 신뢰도 향상
 - 컴퓨터 가격 하락
 - VLSI 출현 $\Rightarrow$ PC 개발

시스템의 분류

1. 개인용 컴퓨터: 소형, 저가
2. 임베디드 컴퓨터
 - 장치 내부에 포함, 장치의 동작 제어
 - 용도에 따라 8비트 64비트 컨트롤러 도입
 - 최소 비용, 요구 성능 만족, 실시간 처리
3. 워크스테이션
4. 슈퍼미니컴퓨터
5. 메인프레임
 - 대용량 저장장치, 대규모 DB 관리/저장 (기관의 빅데이터 관리/저장용)
 - 다중 I/O 채널 $\Rightarrow$ 고속 I/O 처리 능력 보유
6. 슈퍼컴퓨터

구조적 특징에 따른 분류

1. 파이프라인 슈퍼컴퓨터
 - 초기의 슈퍼컴퓨터 구조
 - 슈퍼파이프라인 구조 활용: 고속 벡터 계산
2. 대규모 병렬컴퓨터
 - 수백~수천의 범용 프로세서 상호 연결
3. 클러스터 컴퓨터
 - 네트워크로 연결된 컴퓨터의 집합체
 - 클러스터 미들웨어 이용, 노드(=단위컴퓨터)에 포함된 자원, 단일 시스템 이미지(SSI)로 통합

2. CPU의 구조와 기능

CPU의 기능

- 명령어 인출: 기억장치로부터 명령 로드
- 명령어 해독
- 데이터 인출: 데이터 필요하면 데이터를 로드
- 데이터 처리
- 데이터 저장

구성요소

- ALU
- 산술 연산, 논리 연산 수행

Register

- 액세스 속도가 가장 빠른 기억장치
- CPU 내부에 포함할 수 있는 레지스터 수 제한됨

제어 유닛

- 명령어 해석, 실행 위한 신호 순차 발생

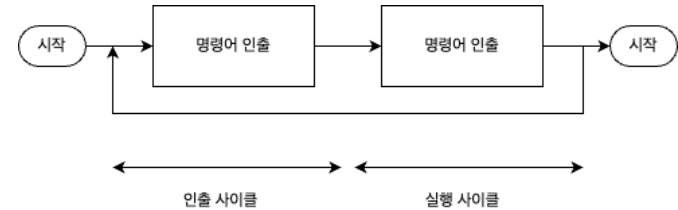
CPU 내부 버스

- ALU와 레지스터 간 데이터 이동을 위한 데이터 선
- + 제어 유닛의 제어 신호 선
- 외부와는 연결되지 않음 버퍼 레지스터/시스템 버스 인터페이스 회로를 통해 시스템 버스와 접속

명령어 실행

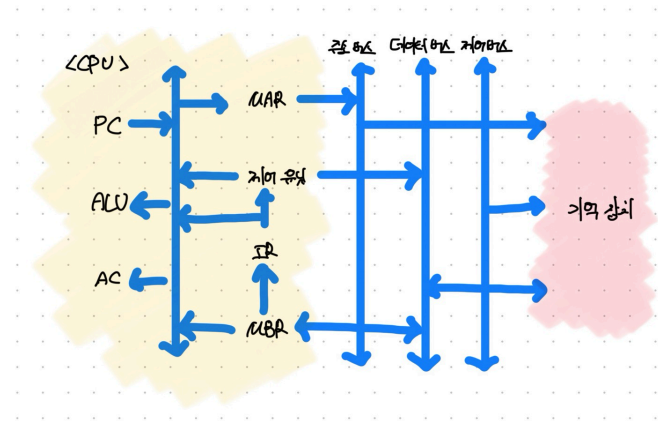
명령어 사이클

- CPU가 한 개 명령어를 실행하는데 필요한 전체 과정
- 프로그램 실행 ⇒ 종료되거나 failure하여 중단될때까지 반복
- 두 개의 서브사이클로 구성



명령어 실행에 필요한 레지스터

- PC; Program Counter
- AC; Accumulator
- 레지스터 길이: CPU가 한 번에 처리할 수 있는 데이터 비트 수 (=word)와 동일
- IR; Instruction Register
- 가장 최근에 인출된 명령어 코드 저장
- MAR; Memory Address Register
- PC에 저장된 명령어 주소, 시스템 주소 버스로 출력되기 전 임시 저장
- MBR; Memory Buffer Register
- 기억장치에 쓰일 데이터/읽은 데이터 임시 저장



Fetch	LOAD addr	STA addr	ADD addr
$t_0 : \text{MAR} \leftarrow \text{PC}$	$t_0 : \text{MAR} \leftarrow \text{IR(addr)}$	$t_0 : \text{MAR} \leftarrow \text{IR(addr)}$	$t_0 : \text{MAR} \leftarrow \text{IR(addr)}$
$t_1 : \text{MBR} \leftarrow \text{M[MAR]}$	$t_1 : \text{MBR} \leftarrow \text{M[MAR]}$	$t_1 : \text{MBR} \leftarrow \text{AC}$	$t_1 : \text{MBR} \leftarrow \text{M[MAR]}$
$\text{PC} \leftarrow \text{PC} + 1$	$t_2 : \text{AC} \leftarrow \text{MBR}$	$t_2 : \text{M[MAR]} \leftarrow \text{MBR}$	$t_2 : \text{AC} \leftarrow \text{AC} + \text{MBR}$
$t_2 : \text{IR} \leftarrow \text{MBR}$		* AC 레지스터 내용 저장	
JUMP addr			
$t_0 : \text{PC} \leftarrow \text{IR(addr)}$			

실행 사이클

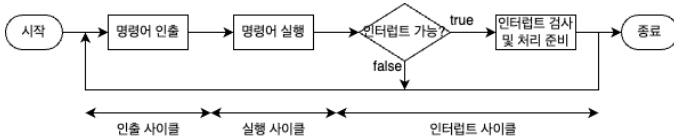
- 해독 ⇒ 수행
- [데이터 이동 | 데이터 처리 | 데이터 저장 | 프로그램 제어]

인터럽트 사이클

- 프로그램 실행 중 현재 처리 중단, 다른 동작 우선 수행
- IRQ ⇒ ISR; Interrupt Service Routine
- 인터럽트 소스 확인 ⇒ 대응 ISR 호출
- 서비스 종료 후, 중단했던 이전 프로그램으로 복귀

인터럽트 사이클 동안 수행

1. 현재 명령 실행 종료 즉시, 다음 실행 명령어의 주소를 스택에 저장
2. ISR 호출: 루틴 시작 주소를 PC에 적재



$t_0 : \text{MBR} \leftarrow \text{PC}$
 $t_1 : \text{MAR} \leftarrow \text{SP};$
 $\text{PC} \leftarrow \text{addr}(\text{ISR.begin})$
 $t_2 : \text{M[MAR]} \leftarrow \text{MBR}$

다중 인터럽트

- ISR 수행 중 인터럽트 또 발생
- 1. 방법#1: ISR 중에는 새로운 IRQ에 대응 않음
 - Interrupt flag = 0 # Interrupt disabled
 - 시스템 핵심적, 중단 불가능한 IO 수행 중 사용
- 2. 방법#2: 인터럽트 우선순위 설정

간접 사이클

- 명령어에 포함된 주소 이용, 필요한 데이터 주소 인출
 ⇒ 간접 주소지정 방식(indirect addressing mode)에서 사용
- 인출 사이클 → 간접 사이클 → 실행 사이클

∴ 인출 내용이 주소

$t_0 : \text{MBR} \leftarrow \text{IR(addr)}$
 $t_1 : \text{MBR} \leftarrow \text{M[MAR]}$
 $t_2 : \text{IR(addr)} \leftarrow \text{MBR}$

명령어 파이프라이닝

- CPU 내부 하드웨어를 여러 단계로 나누어 동시에 여러 명령 처리

Two Stage Instruction Pipeline

- Fetch Stage / Execute Stage 로 분리
- 한 클럭에 동시에 두 명령 처리 가능
- 문제점: 두 단계의 처리 시간이 동일하지 않으면 효율 저하
 ⇒ 각 단계의 처리 시간을 거의 같게 함
 파이프라인 단계를 늘리면 속도 향상

Four Stage Instruction Pipeline

- IF; Instruction Fetch
- ID; Instruction Decode
- OF; Operand Fetch
- EX; Execute

파이프라인에 의한 전체 명령어 실행 시간

k : 파이프라인 단계 수; N : 실행 명령어 수

$$T_k = k + (N - 1) \quad (k > 1)$$
$$T_1 = k * N$$

S_p : Speedup

$$S_p = \frac{T_1}{T_k} = \frac{k \times N}{k \times (n - 1)}$$

파이프라인의 효율 저하

- 모든 명령: 파이프라인 단계 거치는 것이 아님
 - 오퍼랜드를 인출할 필요가 없는 명령도 네 단계를 거쳐야 함
- 파이프라인의 클럭: 가장 오래 걸리는 단계 기준으로 결정

- IF 단계와 OF 단계가 동시에 IO 작업 시 Memory Conflict ⇒ 지연
- 조건 분기 발생 시 처리가 무효화

분기 발생에 의한 성능 저하 개선

- 분기 예측 (Branch Prediction)
 - 분기 발생을 예측하고, 확률에 따라 명령어 인출
- 분기 목적지 선인출 (Prefetch Branch Target)
 - 조건 분기 인식 시, 분기 명령어와 분기 목적지 명령어 모두 인출
 - 조건 결과에 따라 인출한 명령어 선택하여 실행
- 루프 버퍼 (Loop Buffer)
 - IF 단계에 최근 인출된 n 개 명령, 루프 버퍼(고속 기억장치)에 저장
- 지연 분기 (Delayed Branch)
 - 분기 명령의 위치를 재배치하여 지연시킴

상태 레지스터

S	Z	C	X	E	V	I	P
---	---	---	---	---	---	---	---

명령 실행 결과에 따라 조건 플래그 저장

- S 부호 플래그 - 직전 수행된 산술연산 결과의 부호 비트 저장 (양: 0, 음: 1)
- Z 영 플래그 - 연산 결과가 0이면 1true
- C 올림수 플래그 - 덧셈/뺄셈에서 올림수carry, 빌림수borrow 발생시 1true
- E 동등 플래그 - 두 수 비교 결과 세트
- V 오버플로우 플래그 - 오버플로우 발생 여부 세트
- I 인터럽트 플래그 - 인터럽트 가능 여부 세트
- P 슈퍼바이저 플래그 - CPU 실행 모드가 관리자supervisor 모드이면 1, 사용자user 모드이면 0

슈퍼스칼라

- 명령어 파이프라인 자체의 개수를 늘림
- m - 파이프라인의 개수 $\cdots m$ -way 슈퍼스칼라

52p